Lab 3: Integrating and Differentiating Continuous-Time Signals in MATLAB

Habib University
CE-251L /EE-252L Signal and Systems
Prepared by: Dr. Shafayat Abrar and Ms. Hira Mustafa

Objective: Evaluating integrals and derivatives of signals using symbolic and numerical methods.

In-Lab:
Part I: Integrating a signal using symbolic and numerical tools.
Part II: Differentiating a signal using symbolic and numerical tools.
Post Lab:
Part III: Integrating and differentiating basic signal functions like unit-impulse and unit-step.

***Instructions**: If you are doing any analysis, calculation or sketching on paper then attach a **scanned image** of the picture taken; you can either use an application named CamScanner or any other online application to attach a scanned version of the image. **Do not attach raw image only attach the scanned images** in case of any confusion you can ask from any of the RA.*

Useful tip: In MS Word , *in order to check/resolve formatting issues go to the Menu bar Paragraph section you will find* ¶ this symbol*, click on it this will indicate the start of new Paragraph, spaces, tabs, Section breaks and page Breaks.*

Part I: Symbolic and Numerical Integration of a Signal

Here, we learn how to integrate a signal (function) both symbolically and numerically in MATLAB. Consider a signal $x(t)$ which is obtained as the time derivative of another signal $y(t)$,

$$x(t)=\frac{d}{dt}y(t) \quad (1)$$

We witness such relations in electrical and mechanical dynamic systems in a variety of manner. Consider the scenario of a linear electric circuit which contains an inductor and a capacitor. In inductor, the voltage $v_L(t)$ across it is directly proportional to the time rate of change of the current flowing through the inductor, $i_L(t)$, and the constant of proportionality is the inductance L, so

$$v_L(t)=L\frac{d}{dt}i_L(t)$$

Similarly, in a capacitor, the current through the capacitor $i_C(t)$ is equal to capacitance times the time rate of change of capacitor voltage $v_C(t)$, giving

$$i_C(t)=C\frac{d}{dt}v_C(t)$$

Suppose, the current flowing through the inductor is given as (where $f_0 \neq 0$)

$$i_L(t) = A\sin(2\pi f_0 t)$$

The voltage across the inductor may easily be obtained as:

$$v_L(t) = L\frac{d}{dt}i_L(t) = L\frac{d}{dt}A\sin(2\pi f_0 t) = 2\pi f_0 LA\cos(2\pi f_0 t)$$

Using symbolic toolbox, $v_L(t)$ is obtained as follows:

```
syms A f0 t pi L
current = A*sin(2*pi*f0*t);
voltage = L*diff(current,'t')
```

giving

```
voltage =
2*A*L*f0*pi*cos(2*f0*pi*t)
```

On the contrary, given the voltage across the inductor, $v_L(t)$, the current flowing through the inductor is obtained as

$$i_L(t) = \frac{1}{L}\int_{-\infty}^{t} v_L(\lambda)\,d\lambda$$

or, if the initial (reference) time is given to be t_0, then

$$i_L(t) = i_L(t_0) + \frac{1}{L}\int_{t_0}^{t} v_L(\lambda)\,d\lambda$$

where, $i_L(t_0)$ is termed as the initial value of the current, or what you may regard as the *constant of integration*. Note, the variable λ serves as a dummy variable in time. In simple, words, at a given time t, the value of inductor current $i_L(t)$ is obtained by integrating the inductor voltage from the beginning of time (the initial time, t_0) till the given time t, plus the initial value of current $i_L(t_0)$ which happens to depend on the initial condition of the inductor. In principle, $i_L(t_0)$ is

$$i_L(t_0) = \frac{1}{L}\int_{-\infty}^{t_0} v_L(\lambda)\,d\lambda$$

Reverting back to the initial consideration, as expressed in equation (1), given $x(t)$, where $x(t) = \frac{d}{dt}y(t)$, the signal $y(t)$ is obtained by taking integral or anti-derivative of $x(t)$:

$$y(t) = \int_{-\infty}^{t} x(\lambda)\,d\lambda = y(t_0) + \int_{t_0}^{t} x(\lambda)\,d\lambda$$

If $x(t) = 0$ for $t < t_0$, then $y(t_0) = \int_{-\infty}^{t_0} x(\lambda)\,d\lambda = 0$.

Consider $x(t) = A\sin(2\pi f_0 t)$ for $0 \le t \le 3$, $A = 1$ and $f_0 = 1$. Let us integrate the signal $x(t)$ to obtain $y(t) = \int x(t)\,dt$ in MATLAB. Assuming that $x(t) = 0$ for $t < 0$, we get $y(0) = 0$.

Task (a) ☛ Solve the following integral by hand:

$$y(t) = y(0) + \int_0^t x(\lambda)\, d\lambda .$$

Task (b) ☛
Using symbolic tool box, try the following piece of code to integrate $x(t)$ to obtain $y(t)$:

```
syms A f0 t pi
xt = A*sin(2*pi*f0*t);
yt = int(xt,'t')
```

giving

```
yt =
-(A*cos(2*f0*pi*t))/(2*f0*pi)
```

To integrate $x(\lambda)$ with limits from **0** to **t**, try the following code:

```
yt = int(xt,'t',[0,t])
```

Is the value of **yt** obtained above is same as that in task (a)? explain.

Task (c) ☛ Write a MATLAB code to plot $x(t)$ and $y(t)$ for $0 \le t \le 3$. Select time step-size equal to 0.01 second. Obtain both plots on the same figure. Your code should make appropriate use of functions **hold on**; **grid on**; **figure**; **plot**; **legend**; **xlabel**; **ylabel**; and **title.** Show your work to lab demonstrator.

Provide MATLAB code and plot here.

In the following, we learn performing integration operation numerically in MATLAB. Owing to which, the signal $y(t)$ may be computed numerically without requiring to integrate $x(t)$ explicitly either manually or using some symbolic toolbox. The numerical integration is a *cumulative sum* operation where the numerical value of $y(t)$ at time t is obtained as the cumulative sum of $x(t)$ from its initial value $x(0)$ till its value at time t. Conceptually, we have

$$y(t) = y(t_0) + \int_{t_0}^{t} x(\lambda)\, d\lambda \approx y(t_0) + dt \sum_{k=t_0}^{t} x[k]$$

This may easily be done in MATLAB using the built-in function **`cumsum`** as follows:

$$yt = yt0 + dt * cumsum(xt) \quad (2)$$

where **`dt`** is a (small) time step-size as assigned in MATLAB by the user, **`yt0`** is the initial value of $y(t)$ (if there is any), and **`xt`** denotes the discrete values of $x(t)$ as computed in MATLAB. This simplest form of numerical integration is called the rectangular or Euler's integration method.

Task (d) ☛ To understand **`cumsum`**, consider

```
a = [1 2 3 4 5];
```

try

```
b = cumsum(a)
```

What do you notice? Provide your comments below:

Task (e) ☛ Write a MATLAB code to obtain $y(t)$ from $x(t) = A\sin(2\pi f_0 t)$ using **`cumsum`**. Select a time step-size `dt` (like 0.01). Obtain plots of $x(t)$ and $y(t)$ on the same figure. Your code should make appropriate use of functions **hold on**; **grid on**; **figure**; **plot**; **legend**; **xlabel**, **ylabel**; and title. Show your work to lab demonstrator or Instructor.

Provide MATLAB code and plot here.

Task (f) ☛ Compare the numerical values of $y(t)$ as obtained in the MATLAB simulation with the analytical result as obtained earlier in tasks (a) and (b), and note down your observations.

Task (g) ☛ What advantages you may mention of using numerical integration? Think about the difficulty we may have to encounter in solving a complicated integration; think about the real scenarios, where we happen to receive signal in the form of data not in the form of some function; etc.

Task (h) ☛ Solve the following integral numerically using **`cumsum`**:

$$y(t)=\int_0^t x(\lambda)\,d\lambda$$

where $x(t)=sinc(t-6), for\ 0\le t\le 12.$

Note, the (normalized) `sinc` function is defined as:

$$sinc(at)=\frac{\sin(\pi at)}{\pi at}$$

For more information, refer to https://en.wikipedia.org/wiki/Sinc_function.

Assign a time vector from 0 to 12 with a small step-size **`dt`**. Obtain the plot of $x(t)$ using MATLAB built-in **`sinc`** function. Obtain the value of $y(t)$ using **`cumsum`** and plot on the same figure. Your code should make appropriate use of **hold on**; **grid on**; **figure**; **plot**; **legend**; **xlabel**, **ylabel**; and **title.** Select the location of **legend** such that it does not over the traces of the plots. Seek help in MATLAB to learn how **legend** may be located at desirable places.

Provide MATLAB code and plot here.

Part II: Numerical Differentiation of a Signal

Given a signal $y(t)$, another signal $x(t)$ is obtained as time derivative of $y(t)$ as follows:

$$x(t)=\frac{d}{dt}y(t)$$

In MATLAB, this may be done numerically by using the function **diff**. So given the numerical data of signal $y(t)$ in some vector **yt**, the numerical time-derivative of **yt** is obtained as

```
xt = diff(yt) / dt;
```

where **dt** is the time step-size. The function **diff**, when applied on an array, obtains successive differences between consecutive elements of the array. The resulting output vector **xt** becomes smaller in size than the input vector **yt** by one element. To make both vectors (arrays) have same number of elements, either a dummy *zero* or *not-a-number* (nan) element is usually appended (or inserted) in **xt** as follows:

```
xt = [0 xt];
```

or

```
xt = [nan xt];
```

Task (i) ☛ To understand **diff**, consider `a = [1 2 3 4 5];`
and evaluate `b = diff(a)`
What do you notice?

Task (j) ☛ Solve the following differentiation by hand:

$$x(t)=\frac{d}{dt}y(t), \text{where } y(t)=2te^{-t}$$

Task (k) ☛ Write a MATLAB code to plot $y(t)=2te^{-t}$ and the expression of $x(t)$ (as obtained above in task (j)) for $0\le t\le 10$ on a single figure. Select a small-time step-size. Your code should make appropriate use of functions **hold on**; **grid on**; **figure**; **plot; legend; xlabel, ylabel**; and **title**.

Provide MATLAB code here.

Provide MATLAB plot here:

Task (l) ☛ Write a MATLAB code to obtain $x(t)$ numerically from $y(t)=2te^{-t}, 0\le t\le 10$, using the function **`diff`**. Select a reasonable time step-size **`dt`**. Obtain plots of $x(t)$ and $y(t)$ on the same figure. Your code should make appropriate use of functions **hold on**; **grid on**; **figure**; **plot**; **legend**; **xlabel**, **ylabel**; and title.

Provide MATLAB code and plot here.

Task (m) ☛ Solve the following differentiation numerically using function **`diff`**:

$$x(t)=\frac{d}{dt}y(t), \text{where } y(t)=sinc\,(t-6)$$

for $0\le t\le 12$. Select a proper time step-size. Obtain plots of $x(t)$ and $y(t)$ on the same figure. Your code should make appropriate use of functions **hold on**; **grid on**; **figure**; **plot**; **legend**; **xlabel**, **ylabel**; and **title**. Select the location of **legend** such that it does not interfere with the plots. Seek help in MATLAB to learn how location of **legend** may be changed.

Provide your MATLAB code and plot here:

Post Lab

Part III: Integrating and differentiating basic signal functions like unit-impulse and unit-step

Unit-impulse function: The unit-impulse function plays an important role in mathematical modeling and analysis of signals and linear systems. It is defined by the following equations:

$$\delta(t)=\begin{cases}0, t\neq 0\\ undefined, t=0\end{cases}$$

Note that this is an incomplete definition of the function $\delta(t)$ since the amplitude of it is defined only when $t\neq 0$, and is undefined at the time instant $t=0$. This is not possible to draw function $\delta(t)$ in MATLAB due to its incomplete definition. However, it is possible to plot $\delta(t)$ using the following asymptotic alternate definitions:

$$\delta(t)=\frac{\lim_{a\to 0} 1}{a\sqrt{\pi}} e^{-t^2/a^2} \text{ or}$$

$$\delta(t)=\frac{\lim_{a\to 0} a}{(\pi t)^2}\sin^2\left(\frac{\pi t}{a}\right)$$

These definitions also ensure that the area under the curve of $\delta(t)$ is unity, i.e., $\int_{-\infty}^{\infty}\delta(t)dt=1$.

Task (n) ☛ Write a MATLAB code to plot $\delta(t)$ using its alternate asymptotic definitions.

Caution: select a reasonably very small step-size for time axis like **`dt = 0.001`**. Also consider a very small value for a, like **`a = 0.001`**.

Provide MATLAB code and plot here.

Task (o) ☛ Show that the total area under the curve of function $\delta(t)$ as obtained in MATLAB above is **unity**. Hint: sum all values of $\delta(t)$ using the command **`sum`** and multiply it with **`dt`**.

Provide your MATLAB code and result here:

Unit-step function: The unit-step function is useful in situations where we need to model a signal that is turned on or off at a specific time instant. It is defined as follows:

$$u(t)=\begin{cases}1, t>0\\0, t<0\end{cases}$$

The relationship between the unit-step function and the unit-impulse function is important. The unit-step function can be expressed as a running integral of the unit-impulse function:

$$u(t)=\int_{-\infty}^{t}\delta(\lambda)\,d\lambda$$

Task (p) ☛ Integrate the function $\delta(t)$ in MATLAB, as obtained in task (m), and show that it yields $u(t)$. **Hint:** the time integration may easily be implemented using **`cumsum`** and multiplying with **`dt`**.

Provide your MATLAB code and plot here:

Task (q) ☛ Since, $u(t)$ is the integral of $\delta(t)$; therefore, $\delta(t)$ is the time derivative of $u(t)$,

$$\delta(t)=\frac{d}{dt}u(t)$$

Use the MATLAB function **`diff`** in numerical mode, take **`diff`** of expression $u(t)$ as obtained in task (p), divide it with **`dt`**, and show that it yields the unit impulse function

Provide MATLAB code and plot here.

Assessment Rubric
Lab 3: Integrating and Differentiating Continuous-Time Signals in MATLAB

Name:	Student ID:

Marks distribution:

	Tasks	LR2 Code	LR5 Results	LR9 Report	AR4 Report Submission
In-Lab	Task a-b		4	4	/2
	Task c-d	4	8	2	
	Task e-f	4	8		
	Task g-h	4	8		
	Task i-j-k	9	9		
	Task l-m	4	6		
	Task n-o	**6**	6		
	Task p-q	**6**	6		
Total =100		/37	/55	/6	/2
CLO Mapped		**CLO1**	**CLO1**	**CLO1**	**CLO3**

CLO	Total Points	Points Obtained
1	98	
3	2	
Total	**100**	

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

Lab Evaluation Assessment Rubric

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3:Good Points 3	Level 4:Exemplary Points 4
LR2	**Program/ Code/ Simulation Model/ Network Model**	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/ simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR3	**Troubleshooting**	Unable to identify the fault/minimal effort show in troubleshooting.	Able to identify the fault but unable to remove it.	Able to identify the fault but partially removes it.	Able to identify the fault and takes necessary steps and actions to correct it.
LR4	**Data Collection**	Measurements are incomplete, inaccurate and imprecise. Observations are incomplete or not included. Symbols, units and significant figures are not included.	Measurements are somewhat inaccurate and imprecise. Observations are incomplete or vague. Major errors are there in using symbols, units and significant digits.	Measurements are mostly accurate. Observations are generally complete. Minor errors are present in using symbols, units and significant digits.	Measurements are both accurate and precise. Data collection is systematic. Observations are very thorough and include appropriate symbols, units and significant digits and task completed in due time.
LR5	**Results & Plots**	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
LR6	**Calculations**	Formulae used and/or calculations are incorrect. Units are not mentioned with results.	Formulae used are correct but there are few mistakes in calculation which lead to incorrect results.	Formulae used and end results are correct. Complete working steps are not shown. Proper units are not mentioned with results.	Formulae used, end results and all calculations are correct with all the intermediate steps clearly shown. Units are mentioned with results.
LR9	**Report**	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is submitted after due date.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables.
LR11	**Design**	Proposed design is unsubstantiated or does not satisfy most of given	Justification for proposed design is weak, and it only satisfies some	Proposed design is substantiated, preferably through proper analysis,	Proposed design is substantiated, preferably through proper analysis, and

		constraints. The breakdown of system into features is incomplete and still at lower resolution.	constraints. The breakdown of system is missing some features and resolution is not appropriate at some places.	and satisfies most given constraints. The breakdown of system into features seems complete, but resolution is not appropriate at some places.	satisfies all given constraints. The breakdown of system into features seems complete and at appropriate resolution, given students' level.
AR4	***Report Submission**	Not accepted after 1 week.	Late submission after 2 days and within 1 week.	Late submission after the lab timing and within 2 days of the due date.	Timely submission of the report and in the lab time.
AR7	**Report Content/Code comments**	Most of the questions are not answered / figures are not labelled/ titles are not mentioned / units are not mentioned. No comments are present in the code.	Some of the questions are answered, figures are labelled, titles are mentioned and units are mentioned. Few comments are stated in the code.	Majority of the questions are answered, figures are labelled, titles are mentioned and units are mentioned. Comments are stated in the code.	All the questions are answered, figures are labelled, titles are mentioned and units are properly mentioned. Proper comments are stated in the code.